

Allocator-aware library wrappers for dynamic allocation

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Example use cases](#)
4. [Dynamic allocation wrappers](#)
5. [Unique pointers](#)
6. [Design questions](#)
7. [Proposed wording](#)

Revision history

- P00211r0: Initial proposal, pre-Jacksonville.
- D00211r1: make pointer in `allocate_delete` const; delete unnecessary null check from `allocate_new`; add some more design questions.
- P00211r1 (this version): Rename functions as requested by LEWG in Jacksonville. Add conversions to `allocation_deleter`. Add formal wording.

Summary

Short form: We propose to add library functions that allow the systematic use of allocators as a customisation point for dynamic allocations. The new functions complete the following picture:

	using operator {new,delete}	using allocator
Manual	<code>T * p = new T(args...)</code>	<code>auto p = allocator_new<T>(alloc, args...)</code> new!
	<code>delete p</code>	<code>allocator_delete(alloc, p)</code> new!
Unique pointer	<code>default_delete<T></code>	<code>allocation_deleter<T></code> new!
	<code>make_unique<T>(args...)</code>	<code>allocate_unique<T>(alloc, args...)</code> new!
Shared pointer	<code>make_shared<T>(args...)</code>	<code>allocate_shared<T>(alloc, args...)</code>

Long form: The standard library rarely uses `new/delete` directly, but instead allows customisation of dynamic allocation and object construction via allocators. Currently this customisation is only available for container *elements* and for `shared_ptr` (as well as for a few other types that require dynamically allocated memory), but not for the top-level objects themselves.

The proposal is to complete the library facilities for allocator-based customisation by providing a direct mechanism for creating and destroying a dynamically stored object through an allocator, as well as a new deleter type for `unique_ptr` to hold an allocator-managed unique pointee, together with the appropriate factory function.

Example use cases

- Consider an arena allocator. A vector may put its elements into the arena, but the vector itself cannot easily be placed in the same arena. The proposed `std::allocate_unique<std::vector<T, ScopedArenaAlloc<T>>>(arena_alloc)` allows this. (Here we assume the use of the usual alias idiom `template <class T> using ScopedArenaAlloc = std::scoped_allocator_adaptor<ArenaAlloc<T>>;`).

- Consider a shared memory allocator. As in the previous example, containers will put their elements in shared memory, but one process needs to create the container itself and place it in shared memory. This is now possible with `auto p = std::allocator_new<std::vector<T, ScopedShmemAlloc<T>>>(shmem_alloc)`. The returned pointer is presumably an offset pointer, and its offset value needs to be communicated to the other participating processes. The last process to use the container calls `allocator_delete(shmem_alloc, p)`.
- Allocator support is [frequently requested on Stack Overflow](#).

Dynamic allocation wrappers

Allocation and object creation

```
template <class T, class A, class ...Args>
auto allocator_new(A& alloc, Args&&... args) {
    using Traits = allocator_traits<A>;
    static_assert(std::is_same_v<T, typename Traits::value_type>);

    auto p = Traits::allocate(alloc, 1);

    try {
        Traits::construct(alloc, addressof(*p), std::forward<Args>(args)...);
        return p;
    } catch(...) {
        Traits::deallocate(alloc, p, 1);
        throw;
    }
}
```

Object destruction and Deallocation

```
template <class A>
void allocator_delete(A& alloc, typename allocator_traits<A>::const_pointer p) {
    using Traits = allocator_traits<A>;

    Traits::destroy(alloc, addressof(*p));
    Traits::deallocate(alloc, p, 1);
}
```

Unique pointers

To allow `std::unique_ptr` to use custom allocators, we first need a deleter template that stores the allocator:

```
template <class A>
struct allocation_deleter {
    using pointer = typename allocator_traits<A>::pointer;

    A a_; // exposition only

    allocation_deleter(const A& a) : a_(a) {}

    void operator()(pointer p) const {
        allocator_delete(a_, p);
    }
};
```

The factory function is:

```
template <class T, class A, class ...Args>
unique_ptr<T, allocation_deleter<A>> allocate_unique(A& alloc, Args&&... args) {
    return unique_ptr<T, allocator_delete<A>>(allocator_new<T>(alloc, std::forward<Args>
(args)...), alloc);
}
```

The type `T` must not be an array type.

Design questions

- Require exact allocator or allow rebind-copying? Rebind-copying would allow taking the original allocator by const-reference, but would incur a local allocator's destruction (which is not required to be `noexcept`).

Proposed wording

In 20.9.2, add to the synopsis:

```
// 20.9.7, uses_allocator
template <class T, class Alloc> struct uses_allocator;

// 20.9.8, allocator traits
template <class Alloc> struct allocator_traits;

// 20.9.?, allocation helpers:
template <class A, class ...Args>
typename allocator_traits<A>::pointer allocator_new(A& alloc, Args&&... args);
template <class A>
void allocator_delete(A& alloc, typename allocator_traits<A>::const_pointer p);

// 20.9.9?, the default allocator:
template <class T> class allocator;
template <> class allocator<void>;
template <class T, class U>
bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
template <class T, class U>
bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;
```

```
// 20.10.1 class template unique_ptr:
template <class T> struct default_delete;
template <class T> struct default_delete<T[]>;
template <class A> struct allocation_deleter;
template <class T, class D = default_delete<T>> class unique_ptr;
template <class T, class D> class unique_ptr<T[], D>;

template <class T, class... Args> unique_ptr<T> make_unique(Args&&... args);
template <class T> unique_ptr<T> make_unique(size_t n);
template <class T, class... Args> unspecified make_unique(Args&&...) = delete;
template <class T, class A, class ...Args> unique_ptr<T, allocation_deleter<A>>
allocate_unique(A& alloc, Args&&... args)
```

Insert a new subsection between 20.9.8 (allocator traits) and 20.9.9 (the default allocator):

20.9.? Allocation helpers [allocator.alloc_helpers]

The function templates `allocator_new` and `allocator_delete` create and delete objects dynamically using an allocator to provide storage and perform construction (rather than by looking up an allocation function (3.7.4.1)).

```
template <class T, class A, class ...Args>
typename allocator_traits<A>::pointer allocator_new(A& alloc, Args&&... args);
```

Requires: A shall be an allocator class for type T.

Effects: Obtains storage for one element from `alloc` and constructs an object of type T from `args` using `alloc`. If the construction exits with an exception, the storage is released using `alloc`.

Returns: A pointer to the obtained storage that holds the constructed object. [Note: This pointer may have a user-defined type. – end note]

Throws: Any exception thrown by the constructor of T.

```
template <class A>
void allocator_delete(A& alloc, typename allocator_traits<A>::const_pointer p);
```

Requires: p was obtained from an allocator that compares equal to `alloc`, and p is dereferenceable.

Effects: Uses `alloc` to destroy the object *p and to release the underlying storage.

Insert a new subsection 20.10.1.1.? after 20.10.1.1.3 (`default_delete<T[]>`):

20.10.1.1.? allocation_deleter<A> [unique.ptr.dltr.alloc]

```
template <class A>
struct allocation_deleter {
    using pointer = typename allocator_traits<A>::pointer;

    allocation_deleter(const A& alloc);
    template <class B>
    allocation_deleter(const allocation_deleter<B>& other);

    void operator()(pointer p) const;

private:
    A a_; // exposition only
};
```

```
allocation_deleter(const A& alloc);
```

Effects: Initializes `a_` with `alloc`.

```
template <class B> allocation_deleter(const allocation_deleter<B>& other);
```

Effects: Initializes `a_` with `other.a_`.

Remarks: This constructor shall not participate in overload resolution unless `typename allocator_traits<b>::pointer` is implicitly convertible to `pointer`.

```
void operator()(pointer p) const;
```

Effects: Calls `allocator_delete(a_, p)`.

Append a new paragraph to the end of subsection 20.10.1.4 (`unique_ptr` creation):

```
template <class T, class A, class ...Args>
unique_ptr<T, allocation_deleter<A>> allocate_unique(A& alloc, Args&&... args)
```

Remarks: This function shall not participate in overload resolution unless T is not an array.

Returns: unique_ptr<T, allocator_delete<A>>(allocator_new<T>(alloc, std::forward<Args>(args)...), alloc).